Besides the desired *magnitude* array shown in (a), there must be a corresponding *phase* array of the same length. In this example, the phase of the desired frequency response is entirely *zero* (this array is not shown in Fig. 17-1). Just as with the magnitude array, the phase array can be loaded with any arbitrary curve you would like the filter to produce. However, remember that the first and last samples (i.e., 0 and 512) of the phase array must have a value of *zero* (or a multiple of 2π, which is the same thing). The frequency response can also be specified in rectangular form by defining the array entries for the *real* and *imaginary parts*, instead of using the magnitude and phase.

The next step is to take the Inverse DFT to move the filter into the time domain. The quickest way to do this is to convert the frequency domain to rectangular form, and then use the Inverse FFT. This results in a 1024 sample signal running from 0 to 1023, as shown in (b). This is the impulse response that corresponds to the frequency response we want; however, it is not suitable for use as a filter kernel (more about this shortly). Just as in the last chapter, it needs to be *shifted*, *truncated*, and *windowed*. In this example, we will design the filter kernel with $M = 40$, i.e., 41 points running from sample 0 to sample 40. Table 17-1 shows a computer program that converts the signal in (b) into the filter kernel shown in (c). As with the windowed-sinc filter, the points near the ends of the filter kernel are so small that they appear to be zero when plotted. Don't make the mistake of thinking they can be deleted!

```
100 'CUSTOM FILTER DESIGN
110 'This program converts an aliased 1024 point impulse response into an M+1 point
120 'filter kernel (such as Fig. 17-1b being converted into Fig. 17-1c)
130 '
140 DIM REX[1023]                   'REX[ ] holds the signal being converted
150 DIM T[1023]                     'T[ ] is a temporary storage buffer
160 '
170 PI = 3.14159265
180 M% = 40                         'Set filter kernel length (41 total points)
190 '
200 GOSUB XXXX                      'Mythical subroutine to load REX[ ] with impulse response
210 '
220 FOR I% = 0 TO 1023   'Shift (rotate) the signal M/2 points to the right
230   INDEX% = I% + M%/2
240   IF INDEX% > 1023 THEN INDEX% = INDEX%-1024
250   T[INDEX%] = REX[I%]
260 NEXT I%
270 '
280 FOR I% = 0 TO 1023
290   REX[I%] = T[I%]
300 NEXT I%
310 '                               'Truncate and window the signal
320 FOR I% = 0 TO 1023
330  IF I% <= M% THEN REX[I%] = REX[I%] * (0.54 - 0.46 * COS(2*PI*I%/M%))
340  IF I% > M%  THEN REX[I%] = 0
350 NEXT I%
360 '                               'The filter kernel now resides in REX[0] to REX[40]
370 END
```

TABLE 17-1